

THREAT INTELLIGENCE REPORT

SolarWinds SUNBURST Campaign

A CTI-to-Detection-Engineering Case Study

TLP:CLEAR

Prepared by: [Your Name]

Date: September 10, 2026

MITRE ATT&CK Campaign C0024

Scope statement: This report is an independent CTI analysis built entirely on publicly available sources (MITRE ATT&CK, CISA/NSA/FBI advisories, Microsoft, and Mandiant/FireEye reporting). It is not an original investigation and contains no non-public intelligence. Detection logic was validated in a single-host home lab; see Limitations.

Executive Summary

The SolarWinds SUNBURST campaign (MITRE C0024) represents one of the most significant supply-chain compromises in recent history. This report analyzes the campaign's tactics, techniques, and procedures (TTPs) and provides actionable detection engineering guidance for security operations centers (SOCs).

Key Findings

1. The campaign exploited trust at multiple layers: software distribution, authentication, identity, and cloud infrastructure.
2. Behavioral detection provides more durable defensive value than relying exclusively on static indicators of compromise (IOCs) — consistent with the Pyramid of Pain model, under which artifacts tied to attacker behavior (TTPs) are far costlier for an adversary to change than hashes, IPs, or domains.
3. Five key post-compromise behaviors were validated through practical lab testing: suspicious PowerShell execution, scheduled task creation, WMI activity, authentication anomalies, and logging tampering.
4. Detection requires correlation across endpoint, network, identity, and cloud telemetry.
5. Attribution to Russia's SVR is High confidence at the government-assessment level; the public does not have access to all underlying intelligence, so this report treats attribution as informative context rather than an independently verified finding.

This report demonstrates the application of cyber threat intelligence (CTI) analysis to detection engineering, with practical validation in a home lab environment.

Introduction

The SolarWinds supply-chain compromise (also known as SUNBURST) was discovered in December 2020. Attackers inserted malicious code into legitimate Orion software updates, which were then distributed to approximately 18,000 customers. A smaller subset of these victims experienced follow-on post-compromise activity.

This report examines the campaign through a CTI analytical framework, answering seven key intelligence questions (KIQs):

- KIQ 1: What was the Initial Access?
- KIQ 2: What happened after Initial Access?
- KIQ 3: Which TTPs were strategically important?
- KIQ 4: Which IOCs remain useful?
- KIQ 5: How confident is attribution?
- KIQ 6: What can defenders detect?
- KIQ 7: What intelligence gaps remain?

The report follows a structured analytical process: Raw Intelligence → Validated Facts → Assessments → Confidence → Intelligence Gaps → Impact → Detection Opportunities → Lab Validation.

Attack Chain Overview

The diagram below traces the campaign from build-system compromise through to the attackers' ultimate collection and exfiltration objective. It maps each stage to the Validated Facts (F0x) that support it, and flags which stages this report's lab validation actually covers versus which remain outside its scope (e.g., DNS/network telemetry and SAML/cookie abuse were not tested).

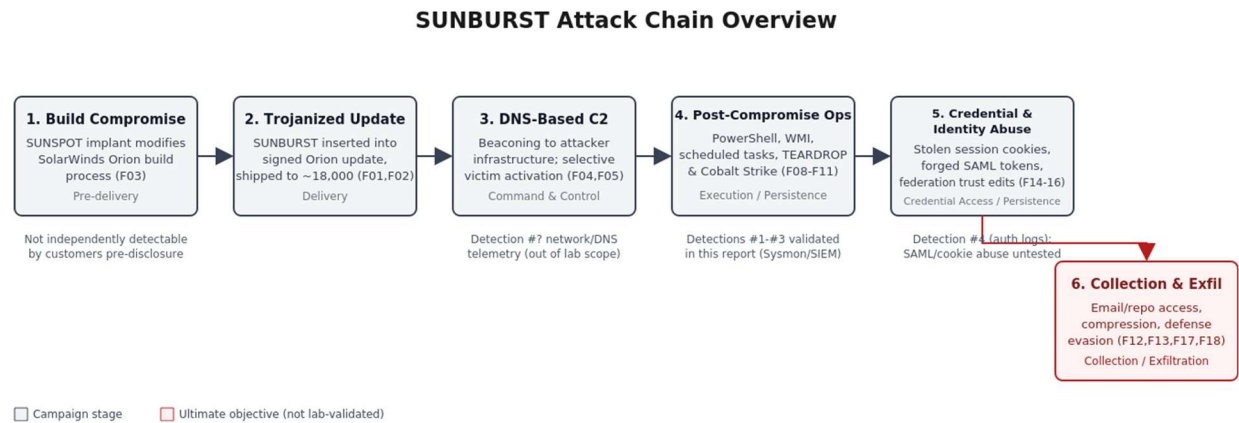


Figure 0: SUNBURST attack chain from build compromise to exfiltration, with lab-validation coverage noted

Validated Facts

The following facts were validated using authoritative sources including MITRE ATT&CK, CISA, NSA, FBI, Microsoft, and Mandiant/FireEye. Confidence here reflects source strength for a directly reported fact, not analytic inference — see the Assessments section for graded analytic judgments built on top of these facts.

ID	Fact	Source Strength	Status	Timeline
F01	APT29 gained access through trojanized SolarWinds Orion updates	Very High	Known	Initial Access
F02	SUNBURST used legitimate SolarWinds code-signing	Very High	Known	Delivery
F03	SUNSPOT modified the build process to insert SUNBURST	Very High	Known	Pre-delivery
F04	SUNBURST used DNS-based C2	High	Known	C2
F05	APT29 used HTTP for post-compromise communications	High	Known	C2
F06	APT29 used compromised credentials for access	High	Known	Credential Access
F07	APT29 performed account discovery	High	Known	Discovery
F08	APT29 used PowerShell	High	Known	Execution
F09	APT29 used WMI for remote execution	High	Known	Lateral Movement
F10	APT29 used scheduled tasks for persistence	High	Known	Persistence
F11	Additional tools (TEARDROP, Cobalt Strike) were deployed	Very High	Known	Post-compromise
F12	APT29 accessed information repositories	High	Known	Collection
F13	APT29 collected email from targeted accounts	High	Known	Collection
F14	Stolen web-session cookies bypassed MFA	Very High	Known	Credential Access
F15	APT29 modified federation trust settings	Very High	Known	Persistence
F16	APT29 used forged SAML tokens	Very High	Known	Credential Access
F17	Collected information was compressed before exfiltration	High	Known	Exfiltration
F18	APT29 attempted to reduce security visibility	High	Known	Defense Evasion
F19	Multiple malware families were used	Very High	Known	Multiple Stages
F20	U.S. government attributed activity to Russia's SVR	Very High	Known	Attribution

Assessments

Based on the validated facts, the following analytic assessments were made. Confidence levels below were recalibrated to distinguish directly-evidenced conclusions from inferences: several judgments originally rated High have been revised to Medium where the underlying evidence supports a plausible conclusion but not a fully verified one (A05, A07). Reserving High confidence for claims with direct technical corroboration — rather than defaulting to it — is itself part of the discipline of analytic writing.

ID	Assessment	Confidence	What Would Change My Mind
A01	Supply-chain compromise was the primary initial access mechanism	High	Evidence of alternative access paths
A02	Attackers exploited trust in legitimate software updates	High	Evidence trust was not relevant
A03	The campaign evolved into a multi-stage intrusion beyond SUNBURST alone	High	Evidence post-compromise activities were unrelated to the initial implant
A04	Attackers did not depend exclusively on SUNBURST for follow-on access	High	Evidence SUNBURST was the only tool used
A05	Compromised credentials were strategically useful for lateral movement and persistence	Medium	Independent evidence credentials were not the primary lateral-movement vector
A06	Identity and cloud authentication abuse were critical to the campaign's impact	High	Evidence cloud/identity access was minimal across victims
A07	Attackers systematically operated through trusted authentication paths rather than exploiting software vulnerabilities post-access	Medium	Evidence of significant vulnerability exploitation post-access
A08	Defense evasion was an important, deliberate part of tradecraft	High	Evidence attackers made no effort to reduce visibility
A09	Behavioral indicators provide more durable defensive value than static IOCs	High	Evidence static IOCs remained effective long-term
A10	Malware signatures alone provide incomplete visibility into this activity	High	Evidence signatures detected all associated activity
A11	Cross-domain (endpoint/network/identity/cloud) detection is necessary for full coverage	High	Evidence single-domain telemetry was sufficient
A12	Trusted-provider compromise creates material downstream risk for customers	High	Evidence provider compromise was effectively isolated
A13	Attribution to Russia's SVR is High confidence at the government-assessment level	High (assessment-level; not independently verifiable)	Public disclosure of underlying government intelligence

Intelligence Gaps

The following intelligence gaps remain:

1. How attackers initially compromised SolarWinds build infrastructure
2. Exact initial-access path for each victim
3. How every compromised credential was originally obtained
4. Scope of stolen session-cookie/SAML abuse across the victim population
5. Exact victim-level attack timelines
6. Complete relationship between all malware/tool components
7. Exact amount and destination of exfiltrated data

These gaps represent the boundaries of public knowledge and highlight the importance of continuous intelligence collection and analysis.

Impact / "So What?"

The SolarWinds campaign demonstrates that a sophisticated intrusion can exploit trust at multiple layers—software distribution, authentication, identity, and cloud infrastructure. Consequently, defenders should not rely solely on malware signatures, software reputation, or password-based authentication monitoring; effective detection requires behavioral correlation across endpoint, network, identity, and cloud telemetry.

This preference for behavioral detection over static indicators maps directly onto David Bianco's Pyramid of Pain: hash values and IP/domain indicators sit at the base of the pyramid and are trivial for an adversary to change, while TTPs sit at the apex and are the most costly for an adversary to alter. The detections built later in this report (PowerShell, scheduled tasks, WMI, authentication patterns, log tampering) were deliberately chosen because they target mid-to-upper-pyramid behaviors rather than the campaign's now-defunct IOCs.

Key Impact Areas

1. **Supply-Chain Compromise:** A trusted software update can become an attack delivery mechanism.
2. **Code-Signing Trust:** A valid signature should not automatically be treated as sufficient proof that software is safe.
3. **Multi-Stage Intrusion:** Removing the initial malware does not necessarily mean the entire intrusion is contained.
4. **Credential Abuse:** Attackers using legitimate credentials can generate activity that resembles legitimate administrative behavior.
5. **Cloud Identity Abuse:** Identity infrastructure itself can become a persistence/access mechanism.
6. **Defense Evasion:** Attackers can intentionally reduce defender visibility.
7. **Static IOC Limitations:** Historical IPs/domains/hashes provide value for retrospective hunting but may become obsolete.
8. **Behavioral Detection:** Behavioral indicators provide more durable detection value.

9. **Cross-Domain Detection:** Endpoint + network + identity + cloud telemetry should be correlated.
10. **Attribution:** Attribution provides strategic context but does not replace detection engineering.

Detection Engineering

Detection Summary Matrix

#	Detection	ATT&CK	Data Source	Confidence
1	Suspicious PowerShell	T1059.001	Sysmon EID 1, PowerShell 4104	High
2	Scheduled Task Creation	T1053.005	Sysmon EID 1, Security 4698	High
3	WMI Activity	T1047	Sysmon EID 1, WMI Logs	High
4	Authentication Anomalies	T1078	Security 4624/4625	High
5	Logging Tampering	T1562.001	Sysmon EID 1, Event 104	High

Detection #1: Suspicious PowerShell (T1059.001)

Threat Intelligence Basis: F08 – APT29 used PowerShell during post-compromise operations. PowerShell is a powerful scripting language frequently abused by attackers for execution, reconnaissance, and lateral movement.

ATT&CK Mapping: T1059.001 – Command and Scripting Interpreter: PowerShell

Detection Hypothesis: Suspicious PowerShell patterns including encoded commands, web requests, and execution bypass can be detected and investigated using endpoint and SIEM telemetry.

Telemetry Required: Sysmon Event ID 1 (Process Creation), PowerShell Operational Log Event ID 4104

Lab Simulation

```
powershell -Command "Invoke-WebRequest -UseBasicParsing https://example.com"
```

Rationale: This command mimics the behavior of downloading second-stage payloads, consistent with APT29's post-compromise activities (F05, F11).

Telemetry Verification

Before testing the detection, the data pipeline was confirmed operational (Sysmon process creation events flowing to Splunk from a Windows 10 endpoint).

Detection Query Results

The combined detection query identified all suspicious patterns:

```
index=* EventCode=1 Image="*powershell.exe"
( CommandLine="*-e *" OR CommandLine="*-EncodedCommand *"
  OR CommandLine="*Invoke-WebRequest*"
  OR CommandLine="*.DownloadString*"
  OR CommandLine="*-ExecutionPolicy Bypass*"
  OR CommandLine="*-WindowStyle Hidden*" )
| table _time, host, User, Image, CommandLine, ParentImage
| sort - _time
```


Investigation

One encoded command was decoded for analysis:

```
index=* EventCode=1 CommandLine="*-e V3JpdGUtSG9zdCAiQ1RJIFBvcnRmb2xpbyBUZXN0Ig=="
| eval DecodedCommand=base64decode("V3JpdGUtSG9zdCAiQ1RJIFBvcnRmb2xpbyBUZXN0Ig==")
| table _time, host, User, CommandLine, DecodedCommand
```

Result: The command decoded to Write-Host "CTI Portfolio Test", confirming this was a harmless test command.

Assessment: While these commands were benign tests, the detection patterns are consistent with attacker behavior documented in the SolarWinds campaign. In a production environment, Base64-encoded PowerShell and web requests would warrant immediate investigation.

Tuning Considerations (False Positives)

As written, this query would generate significant noise in most enterprise environments: Invoke-WebRequest, .DownloadString, and even encoded commands are routinely used by legitimate configuration-management tools, software installers, and admin scripts (e.g., SCCM, Intune, Chocolatey, CI/CD agents). Recommended tuning before production deployment:

- Baseline normal PowerShell usage per host/role for 1–2 weeks before enabling alerting, to establish a known-good parent-process and command-line population.
- Exclude known-good parent processes (e.g., recognized RMM/patch-management agents) rather than suppressing the technique entirely.
- Prioritize alerts where PowerShell spawns from an unusual parent (Office apps, browsers, scripting hosts) rather than from expected admin tooling — parent-process context does more to separate signal from noise here than the command-line pattern alone.
- Treat Base64-decoded content as a secondary enrichment step, not a standalone alerting trigger, since encoding alone is not inherently malicious.

Detection #2: Scheduled Task Creation (T1053.005)

Threat Intelligence Basis: F10 – APT29 used scheduled tasks in post-compromise activity, including persistence and lateral movement.

ATT&CK Mapping: T1053.005 – Scheduled Task/Job: Scheduled Task

Detection Hypothesis: Scheduled task creation can be detected through Sysmon process creation events for `schtasks.exe`, and suspicious patterns can be identified through task naming and command-line parameters.

Telemetry Required: Sysmon Event ID 1 (Process Creation), Windows Security Log Event ID 4698

Lab Simulation

```
schtasks /create /tn "CTI_Lab_Test" /tr "C:\Windows\System32\calc.exe" /sc ONCE /st 12:00 /ru SYSTEM
```

Detection Query Results

Scheduled task creation was detected via Sysmon:

```
index=* EventCode=1 Image="*schtasks.exe" CommandLine="*/create*"
| table _time, host, User, Image, CommandLine, ParentImage
| sort - _time
```

Investigation

Task details were verified through investigation:

```
index=* TaskName="CTI_Lab_Test"
| table _time, host, EventCode, User, TaskName, Command
| sort - _time
```

Assessment: The task was created as a test and subsequently deleted. In a real intrusion, scheduled tasks are commonly used for persistence, and randomly named tasks with suspicious execution paths would warrant immediate investigation.

Tuning Considerations (False Positives)

`schtasks /create` fires constantly in normal enterprise operations — software installers, backup jobs, and IT automation all create scheduled tasks routinely. To make this detection production-viable:

- Filter to tasks running as SYSTEM or a privileged account combined with an unusual `/tr` target (e.g., PowerShell, `cmd.exe`, LOLBins, or a binary in a user-writable path like `%TEMP%` or `%APPDATA%`).
- Flag task names that are random-looking, mimic legitimate Windows task names, or were created outside a maintenance window.
- Correlate task creation with the parent process — tasks created by an interactive PowerShell or Office-spawned process are far higher risk than those created by a signed installer.

Detection #3: WMI Activity (T1047)

Threat Intelligence Basis: F09 – APT29 used WMI for remote execution and lateral movement.

ATT&CK Mapping: T1047 – Windows Management Instrumentation

Detection Hypothesis: WMI activity, particularly process creation via WMI, can be detected through Sysmon process creation events.

Telemetry Required: Sysmon Event ID 1 (Process Creation), Windows WMI-Activity Logs

Lab Simulation

```
wmic process call create "calc.exe"
```

Detection Query Results

WMI activity was detected via Sysmon:

```
index=* EventCode=1 Image="*wmic.exe"
| table _time, host, User, Image, CommandLine, ParentImage
| sort - _time
```

Assessment: WMI is a powerful tool frequently abused by attackers. The wmic process call create command is particularly suspicious as it indicates process creation through WMI, often used for lateral movement. In a production environment, this activity would warrant correlation with other indicators and investigation of the parent process.

Detection #4: Authentication Anomalies (T1078)

Threat Intelligence Basis: F06 – APT29 used compromised credentials to access systems and support lateral movement.

ATT&CK Mapping: T1078 – Valid Accounts

Detection Hypothesis: Unusual authentication patterns, including remote logins and failed logins, can be detected through Windows Security Logs.

Telemetry Required: Windows Security Log Event ID 4624 (Successful Logon), Event ID 4625 (Failed Logon)

Lab Simulation: Remote Desktop login from Ubuntu to Windows 10

Detection Query Results

Remote authentication was detected in Security Logs:

```
index=* EventCode=4624 Logon_Type=10
| table _time, host, Account_Name, Workstation_Name, Logon_Type, Source_Network_Address
| sort - _time
```

Assessment: Remote logins (Logon Type 10) are common for both legitimate administrators and attackers. In a production environment, this activity should be correlated with:

- Time of day (outside business hours = more suspicious)

- Source IP (known vs. unknown)
- Account (privileged accounts are higher risk)
- Other activity from the same user/host

Detection #5: Logging Tampering (T1562.001)

Threat Intelligence Basis: F18 – APT29 attempted to reduce security visibility by interfering with security monitoring/logging.

ATT&CK Mapping: T1562.001 – Impair Defenses: Disable or Modify Tools

Detection Hypothesis: Attempts to tamper with logging, including clearing logs or using logging tools, can be detected through Sysmon process creation events.

Telemetry Required: Sysmon Event ID 1 (Process Creation), Windows Event Log Event ID 104 (Log Cleared)

Lab Simulation

```
wevtutil get-log System
```

Note: This is a safe query command that does not actually clear logs.

Detection Query Results

Logging tool usage was detected via Sysmon:

```
index=* EventCode=1 Image="*wevtutil.exe"
| table _time, host, User, Image, CommandLine
| sort - _time
```

Assessment: wevtutil.exe is a legitimate Windows tool for managing event logs. However, its usage with commands like cl (clear) or disable is highly suspicious. In a production environment, any usage of wevtutil with destructive commands would warrant immediate investigation.

Lab Validation Results

The lab validation successfully demonstrated detection of five key behaviors documented in the SolarWinds campaign:

Detection	Test Performed	Result	Key Finding
PowerShell	Invoke-WebRequest to example.com	☑ Detected	Web requests in PowerShell are visible and should be monitored
Scheduled Task	Created CTI_Lab_Test task	☑ Detected	Task creation is visible through Sysmon
WMI	wmic process call create	☑ Detected	WMI process creation is detectable
Authentication	RDP login from Ubuntu	☑ Detected	Remote logins are captured in Security Logs
Logging Tampering	wevtutil query command	☑ Detected	Logging tool usage is visible through Sysmon

Key Learnings

1. Sysmon provides critical visibility into process creation and command-line activity.

2. Windows Security Logs are essential for authentication monitoring but may require audit policy configuration.
3. PowerShell logging (ScriptBlock) provides deep visibility into executed commands.
4. Behavioral detection is more durable than static IOC-based detection.
5. Correlation across multiple data sources provides the most complete picture.

Limitations of Lab Validation

1. No actual malware execution — testing simulated post-compromise behaviors only.
2. Limited environment — single-host lab without enterprise complexity.
3. No network telemetry — Zeek/Suricata not integrated.
4. No cloud environment testing — SAML/cookie abuse not validated.

Recommendations

For SOC Defenders

1. Enable Comprehensive Logging

- Enable Sysmon with a robust configuration
- Enable PowerShell ScriptBlock logging
- Enable security audit policies for logon, object access, and policy changes

2. Implement Behavioral Detection

- Monitor for encoded PowerShell commands (-e, -EncodedCommand)
- Detect Invoke-WebRequest and .DownloadString in PowerShell
- Alert on schtasks /create and wmic process call create, tuned per the false-positive guidance above
- Monitor for authentication anomalies (time, location, frequency)

3. Implement Correlation

- Correlate endpoint, network, identity, and cloud telemetry
- Use a SIEM for central visibility and alerting
- Build dashboards for suspicious activity patterns

4. Focus on Identity Protection

- Implement MFA
- Monitor for token and session cookie abuse
- Audit federation trust settings
- Monitor for privileged account usage

5. Develop Incident Response Playbooks

- Playbook for suspicious PowerShell activity
- Playbook for credential abuse
- Playbook for scheduled task creation
- Playbook for logging tampering attempts

For Threat Intelligence

1. Continuously Monitor

- Track APT29 TTPs and infrastructure
- Monitor for new intelligence gaps being filled
- Track evolving detection opportunities

2. Share Intelligence

- Participate in ISACs and information-sharing groups

- Share IOCs and behavioral indicators
- Contribute to open-source intelligence (OSINT)

3. Update ATT&CK Mappings

- Regularly review and update TTP mappings
- Track new techniques and sub-techniques
- Map detection to MITRE ATT&CK

Limitations

This report and its findings have the following limitations:

1. No Actual Malware Reproduction

- The lab validation did not reproduce the actual SUNBURST malware
- No malicious code was executed
- Testing simulated post-compromise behaviors only

2. Limited Environment

- Testing occurred in a single-host lab environment
- Did not include full enterprise complexity
- Did not include network telemetry (Zeek/Suricata)
- Did not include cloud environment testing

3. Limited Scope

- Only 5 of many SolarWinds TTPs were tested
- Supply-chain compromise itself was not reproduced
- Attribution analysis is based on government reports, not independent assessment

4. Telemetry Limitations

- Some event logs may not be available in all environments
- Audit policies may vary across organizations
- SIEM configuration differences may impact detection

5. False Positive Considerations

- Detection tuning requires production-specific baselining
- The false-positive guidance in this report is analytic reasoning, not empirical measurement against real enterprise traffic
- Risk-based scoring would require additional development

References

Primary Sources

1. MITRE ATT&CK. (2023). Campaign C0024 – SolarWinds Compromise. attack.mitre.org/campaigns/C0024/
2. MITRE ATT&CK. (2023). Software S0559 – SUNBURST. attack.mitre.org/software/S0559/
3. MITRE ATT&CK. (2023). Software S0562 – SUNSPOT. attack.mitre.org/software/S0562/
4. MITRE ATT&CK. (2023). Technique T1553.002 – Code Signing. attack.mitre.org/techniques/T1553/002/
5. CISA, NSA, & FBI. (2021, April 15). Russian SVR Targeting US and Allied Networks. cisa.gov/news-events/alerts/2021/04/15/nsa-cisa-fbi-joint-advisory-russian-svr-targeting-us-and-allied
6. CISA. (2021). SolarWinds and Active Directory/M365 Detection Resources. cisa.gov
7. Microsoft. (2020). SolarWinds/Solorigate Technical Analyses.
8. Mandiant/FireEye. (2020). SolarWinds Supply Chain Attack Analysis.
9. SolarWinds. (2020). Security Advisory.

Additional Resources

1. National Security Agency. (2021). Cybersecurity Information Sheets.
2. CrowdStrike. (2021). SUNSPOT and SUNBURST Technical Analysis.
3. Splunk. (2024). Security Detection and Investigation Documentation.
4. Microsoft Sysinternals. (2024). Sysmon Documentation.
5. Bianco, D. (2014). The Pyramid of Pain. detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html